FTEC5660 Homework 2 – Part 1

CV Verification Agent System

Based on DeepSeek-Chat + SocialGraph MCP Tools

Student ID: 1155249660 | Language: Python 3.11 | Framework: LangChain 1.0

1. System Architecture & Design Decisions

The CV Verification Agent is a multi-tool autonomous agent that cross-references a candidate's CV against LinkedIn and Facebook public profiles via an MCP (Model Context Protocol) server. The system consists of three layers:

Component	Detail
LLM Core	DeepSeek-Chat (via OpenAI-compatible API, temperature=0.0)
Tool Interface	SocialGraph MCP Server at ftec5660.ngrok.app/mcp
Agent Framework	LangChain 1.0 – bind_tools() + manual tool dispatch loop
CV Parser	MarkItDown (primary) + PyPDF2 (fallback) for PDF text extraction
Output	Structured verification report + float score in [0, 1]

Key Design Decisions

- **Temperature = 0.0** – Ensures deterministic, consistent evaluation across CVs.
- **Max 20 agent turns per CV** – Balances thoroughness with API cost; sufficient for 4–5 tool calls.
- **Tool retry (3× with 2 s delay)** – Guards against transient MCP server errors.
- **Score extraction via regex** – Parses the mandatory *VERIFICATION_SCORE: X.XX* line from LLM output.
- **Strict system prompt** – Enforces a 7-category discrepancy framework and exact report structure.

2. Agent Workflow & Tool-Use Strategy

For each CV the agent follows a deterministic 6-step investigation pipeline:

Step	Action
Step 1	Extract CV fields (name, location, employer, education, skills)
Step 2	search_linkedin_people – find the LinkedIn profile by name + location
Step 3	get_linkedin_profile – retrieve full work history, education, skills
Step 4	search_facebook_users – find the Facebook profile by name

Step	Action
Step 5	get_facebook_profile – retrieve employment, education, hometown
Step 6	Generate structured report with 7-category discrepancy analysis + score

Tool-Use Strategy

- Tools are bound to the LLM with **bind_tools()** so DeepSeek can decide *when* to call them.
- The agent always searches LinkedIn first (more structured data) before Facebook.
- If the exact-match LinkedIn search fails, the agent retries with relaxed parameters.
- Tool results are appended as **ToolMessage** objects, maintaining a complete chat history.
- The loop terminates when the LLM produces a turn with *no tool calls* (final report).

Core Agent Loop (simplified)

```
messages = [SystemMessage(SYSTEM_PROMPT), HumanMessage(cv_text)] for turn in
range(MAX_TURNS): response = llm_with_tools.invoke(messages) messages.append(response)
if response.tool_calls: for tc in response.tool_calls: result = await
tool_map[tc.name].ainvoke(tc.args) messages.append(ToolMessage(result,
tool_call_id=tc.id)) else: return extract_score(response.content), response.content
```

3. Sample Verification Results

Summary Table

CV	Name	Score	Decision	Key Findings
CV_1	John Smith	0.70	**Valid (>0.5)**	LinkedIn match; Facebook shows different employment
CV_2	Minh Pham	0.75	**Valid (>0.5)**	Strong LinkedIn match; Facebook has employment/education gaps
CV_3	Wei Zhang	0.75	**Valid (>0.5)**	Core claims verified; minor hometown & YoE discrepancies
CV_4	Rahul Sharma	0.15	**Suspicious (≤0.5)**	Severe mismatches – different employers, degrees, timeline overla
CV_5	Rahul Sharma	0.15	**Suspicious (≤0.5)**	CV companies not found; education institution & dates conflict

Score Interpretation

Scores above 0.5 are classified as **Valid**; scores at or below 0.5 are **Suspicious**. CV_1–CV_3 (0.70–0.75) have minor cross-platform inconsistencies typical of real profiles where Facebook data is less maintained. CV_4 and CV_5 (0.15) exhibit fabricated employment histories, impossible timeline overlaps, and mismatched education credentials – clear red flags for potential fraud.

```
Final scores = [0.70, 0.75, 0.75, 0.15, 0.15] | Decisions = [Valid, Valid, Valid, Suspicious, Suspicious]
```

4. Implementation Notes

Dependencies

Package	Purpose
langchain==1.0.2	Agent framework
langchain-openai==1.0.1	DeepSeek/OpenAI LLM binding
langchain-mcp-adapters	MCP tool client
markitdown	PDF → Markdown text extraction
PyPDF2	Fallback PDF parser
python-dotenv	Environment variable management

File Structure

- **homework2_part1.py** – main agent implementation (487 lines)
- **sample_cvs/CV_1.pdf … CV_5.pdf** – input CVs
- **verification_reports/CV_X_report.txt** – full per-CV reports saved automatically

Limitations

- MCP server availability depends on ngrok tunnel uptime.
- LLM non-determinism: temperature=0 minimises but does not eliminate variation.
- Facebook matching is fuzzy – common names may yield false-positive profiles.